

SNIX Synchronous FIFO — Verification Test Plan

Document version 0.2 (registered read port) | DUT: snix_sync_fifo | Author: Sajal Osta

1. DUT description and configuration

snix_sync_fifo is a single-clock synchronous FIFO with a registered read port. Configuration under test: DATA_WIDTH = 32, FIFO_DEPTH = 16. Reset (rst_n) is asynchronous, active-low.

Signal	Dir	Width	Description
clk	in	1	Clock. All state updates on posedge.
rst_n	in	1	Async active-low reset. Clears pointers; rd_data = 0.
wr_en	in	1	Write enable. Accepted when full = 0.
wr_data	in	32	Write data, stored on the accepting edge.
full	out	1	Occupancy = 16. Blocks further writes.
rd_en	in	1	Read enable. Accepted when empty = 0.
rd_data	out	32	Registered read data. Valid the cycle after an accepted read; holds until the next accepted read.
empty	out	1	Occupancy = 0. Blocks further reads.

1.1 Write operation

Pins involved: wr_en, wr_data, full, clk.

Protocol:

1. Check full = 0 (writes are not accepted while full = 1).
 2. Drive wr_data with the word to store and assert wr_en = 1.
 3. On the next posedge of clk the word is stored and the write pointer increments. full asserts on this edge if this was the 16th occupying write.
 4. Deassert wr_en (or keep it high with new wr_data for back-to-back writes, one word per cycle).
- A write attempted while full = 1 is ignored: no store, no pointer change, no flag change.

1.2 Read operation

Pins involved: rd_en, empty, rd_data, clk.

Protocol:

1. Check empty = 0 (reads are not accepted while empty = 1).
2. Assert rd_en = 1 for one clock cycle. This is a request to pop the oldest word.
3. On the next posedge of clk the read is accepted: the oldest word is captured into rd_data and the read pointer increments. empty asserts on this edge if this was the last stored word.
4. Sample rd_data in the following cycle (read data latency = 1 cycle from the accepting edge).
5. rd_data holds this value until the next accepted read, including after the FIFO is empty. For back-to-back reads, keep rd_en = 1: one word pops per cycle and rd_data streams the sequence, each value one cycle behind its accepting edge.

A read attempted while empty = 1 is ignored: no pointer movement, rd_data unchanged, no flag change.

1.3 FULL condition

The FIFO is full when occupancy = 16 (FIFO_DEPTH): every memory slot holds an unread word and there is no room for another write. While full = 1 all writes are blocked.

Detection: the design uses 5-bit pointers for a 16-deep memory - 4 address bits plus one extra wrap bit (MSB) that flips each time a pointer wraps past slot 15. The FIFO is full when the two pointers address the same slot but are on opposite wrap phases: $wr_ptr[4] \neq rd_ptr[4]$ AND $wr_ptr[3:0] == rd_ptr[3:0]$. In other words, the writer has lapped the reader exactly once around the buffer. full asserts on the clock edge that commits the 16th occupying write and deasserts on the next accepted read.

1.4 EMPTY condition

The FIFO is empty when occupancy = 0: there is no unread word to deliver. While empty = 1 all reads are blocked.

Detection: the FIFO is empty when the two pointers are identical in all 5 bits: $wr_ptr == rd_ptr$ - same slot AND same wrap phase, meaning the reader has consumed exactly as many words as the writer has produced. After reset both pointers are 0, so the FIFO comes up empty. empty asserts on the clock edge that pops the last stored word and deasserts on the edge that commits a write into an empty FIFO.

Note the two conditions differ only in the wrap bit: equal low bits with equal MSBs means empty; equal low bits with opposite MSBs means full. This is why the extra pointer bit exists. A simultaneous accepted read + write leaves occupancy unchanged, so neither flag can assert from it.

1.5 Reset state

After rst_n assertion: empty = 1, full = 0, rd_data = 0x00000000.

2. DUT features

The features of snix_sync_fifo that this plan verifies. Each test section names the feature it exercises.

F1 — Reset state

The asynchronous active-low reset initializes the FIFO to a known state: empty = 1, full = 0, rd_data = 0x00000000, and both pointers cleared. Activity during reset has no effect on state. *Tested by: T1.*

F2 — Write/read datapath

Words presented on wr_data are stored on their accepting clock edge and later delivered on rd_data unmodified: every bit written is the bit read. *Tested by: T2, T9.*

F3 — Registered read latency

The read port is registered: an accepted read delivers its word on rd_data exactly one cycle after the accepting edge, and rd_data holds its value between accepted reads, including after the FIFO drains to empty. *Tested by: T2, T5, T9.*

F4 — full flag

full = 1 exactly while occupancy = 16: it asserts on the clock edge of the 16th occupying write and deasserts on the next accepted read. It is never asserted at any lower occupancy. *Tested by: T3.*

F5 — Overflow protection

A write attempted while full = 1 is ignored completely: no word is stored, no pointer moves, no flag changes, and the attempted value never appears on any later read. *Tested by: T4.*

F6 — FIFO ordering

Data reads out in exactly the order it was written and in exactly the same quantity: every word written is read out once and only once — nothing is lost, duplicated, created, modified, or reordered — so the count read equals the count written, under any legal sequence of operations. *Tested by: T5, T7, T8.*

F7 — empty flag

empty = 1 exactly while occupancy = 0: it deasserts on the edge that commits a write into an empty FIFO and asserts on the edge that pops the last stored word. *Tested by: T5, T6.*

F8 — Underflow protection

A read attempted while empty = 1 is ignored completely: no pointer movement, no flag change, and rd_data does not change. *Tested by: T6.*

F9 — Pointer wraparound

Data integrity holds when the 5-bit pointers wrap past slot 15 and reuse memory locations: post-wrap writes and reads behave identically to pre-wrap ones. *Tested by: T7.*

F10 — Simultaneous read/write

A write and a read accepted on the same clock edge both take full effect: occupancy is unchanged, both flags hold, and ordering is preserved. *Tested by: T8.*

3. Test case summary

ID	Test name	Purpose
T1	cold_start	Reset brings FIFO to known empty state
T2	one_in_one_out	Basic write, registered read timing
T3	fill_sixteen	full asserts exactly at depth 16
T4	seventeenth_write	Write while full is ignored

T5	full_drain	Data order and empty at drain
T6	read_on_dry	Read while empty is ignored
T7	lap_the_buffer	Pointer wrap data integrity
T8	push_pull_together	Concurrent read+write at half fill
T9	four_beat_burst	4-deep burst data integrity

4. Test cases

4.1 » T1 — cold_start

Feature under test: F1 — Reset state: asynchronous active-low reset brings the FIFO up empty (empty=1, full=0, rd_data=0x0) with both pointers cleared.

Description: Verify that asynchronous reset initializes the FIFO to a known empty state.

Preconditions: None. Test starts from power-on.

Stimulus: Drive rst_n = 0 for 3 clock cycles with wr_en = 0 and rd_en = 0, then release rst_n = 1 and wait 1 cycle.

Expected results:

- empty = 1 immediately after reset release.
- full = 0 immediately after reset release.
- rd_data = 0x00000000 immediately after reset release.
- No write or read side effects occur during reset.

Pass criteria: Test passes if empty = 1, full = 0 and rd_data = 0x00000000 after reset release.

4.2 » T2 — one_in_one_out

Feature under test: F2 — Write/read datapath: a word written is stored and later delivered on rd_data unmodified. F3 — Registered read latency: rd_data shows the popped word one cycle after the accepted read.

Description: Verify a single write followed by a single read with registered read timing.

Preconditions: FIFO is empty after reset (T1 state).

Stimulus: Write one word with wr_data = 0xA5A50001, wr_en = 1 for exactly 1 clock cycle. Wait 1 cycle. Assert rd_en = 1 for 1 cycle. Sample rd_data one cycle after the read.

Expected results:

- empty = 0 on the clock edge that commits the write.
- rd_data remains 0x00000000 before any read is requested (registered read: no data before rd_en).
- One cycle after the accepted read, rd_data = 0xA5A50001.
- empty = 1 on the clock edge that commits the read.
- rd_data continues to hold 0xA5A50001 after the FIFO is empty.
- full = 0 throughout the test.

Pass criteria: Test passes if the written word appears on rd_data exactly one cycle after the read and empty returns to 1.

4.3 » T3 — fill_sixteen

Feature under test: F4 — Full flag / FULL condition: full asserts on the clock edge of the 16th occupying write (occupancy = 16 = FIFO_DEPTH) and blocks further writes while high.

Description: Verify that full asserts exactly when the 16th word is written, not earlier.

Preconditions: FIFO is empty.

Stimulus: Write 16 words back-to-back with wr_en = 1, wr_data = i for i = 0 to 15 (one word per clock cycle).

Expected results:

- empty = 0 from the edge committing write number 1.
- full = 0 after write number 15 (occupancy 15).
- full = 1 on the clock edge that commits write number 16 (occupancy 16).
- No write is lost: a subsequent drain returns all 16 values.

Pass criteria: Test passes if full asserts exactly at write number 16 and not at write number 15.

4.4 » T4 — seventeenth_write

Feature under test: F5 — Overflow protection: a write attempted while full=1 is ignored — no store, no pointer change, no flag change, data never overwritten.

Description: Verify that a write attempted while full = 1 is ignored and does not corrupt contents.

Preconditions: FIFO contains 16 words: values 0 to 15 (T3 end state), full = 1.

Stimulus: Attempt one additional write with wr_data = 0xDEADBEEF, wr_en = 1 for 1 cycle while full = 1. Then drain all 16 words with rd_en = 1, sampling rd_data one cycle after each read.

Expected results:

- full remains 1 during and after the blocked write attempt.
- Occupancy is unchanged by the blocked write.
- The drain returns exactly the values 0,1,2,...,15 in order.
- The value 0xDEADBEEF does not appear anywhere in the drained data.

Pass criteria: Test passes if the 17th write leaves contents and flags unchanged.

4.5 » T5 — full_drain

Feature under test: F6 — FIFO ordering: words drain out in exact write order. F7 — Empty flag / EMPTY condition: empty asserts on the read that consumes the last stored word (occupancy 0). F3 — Registered read latency.

Description: Verify FIFO ordering and that empty asserts exactly when the last word is read.

Preconditions: FIFO contains 16 words: values 0 to 15, full = 1.

Stimulus: Hold rd_en = 1 for 16 consecutive cycles. Sample rd_data one cycle after each read edge.

Expected results:

- The 16 sampled values are 0,1,2,...,15 in FIFO order, each valid one cycle after its read edge.
- full = 0 on the clock edge that commits read number 1.
- empty = 0 after read number 15 (occupancy 1).
- empty = 1 on the clock edge that commits read number 16 (occupancy 0).
- After the drain, rd_data holds the last value read (15).

Pass criteria: Test passes if all 16 values match in order and empty asserts exactly at read number 16.

4.6 » T6 — read_on_dry

Feature under test: F8 — Underflow protection: a read attempted while empty=1 is ignored — no pointer movement, rd_data holds its last value. F7 — Empty flag stays asserted while occupancy is 0.

Description: Verify that a read attempted while empty = 1 is ignored: no pointer movement and no rd_data change.

Preconditions: FIFO is empty after reset. rd_data = 0x00000000.

Stimulus: Assert rd_en = 1 for 3 consecutive cycles while empty = 1. Then write one word wr_data = 0x00000077, wait 1 cycle, read it with rd_en = 1 for 1 cycle, and sample rd_data one cycle later.

Expected results:

- empty remains 1 during all 3 blocked read attempts.
- rd_data remains 0x00000000 during and after the blocked read attempts (unchanged).
- After the follow-up write and accepted read, rd_data = 0x00000077 one cycle after the read edge.
- empty = 1 after the final read.
- full = 0 throughout the test.

Pass criteria: Test passes if blocked reads cause no state change and the follow-up read returns 0x00000077.

4.7 » T7 — lap_the_buffer

Feature under test: F9 — Pointer wraparound: write/read pointers lap the 16-deep memory and addressing stays correct past the wrap. F6 — FIFO ordering and count preserved across the lap: every word written is read out once and only once, count out equals count in, nothing lost, duplicated, or created.

Description: Verify data integrity across write/read pointer wraparound.

Preconditions: FIFO is empty. Pointers advance by 16 during phase 1 of this test.

Stimulus: Phase 1: write 16 words (values 0 to 15), then drain all 16 with rd_en = 1, sampling one cycle after each read. Phase 2: write 8 more words with wr_data = 0x1000 + i for i = 0 to 7 (pointers wrap past 16), then drain all 8 the same way.

Expected results:

- Phase 1 drain returns 0,1,...,15 in order and ends with empty = 1.
- Phase 2 drain returns 0x1000,0x1001,...,0x1007 in order, each sampled one cycle after its read edge.
- empty = 1 after the phase 2 drain; full = 0 throughout phase 2.
- No data corruption occurs at the pointer wrap boundary.

Pass criteria: Test passes if all 8 post-wrap values match written data in order.

4.8 » T8 — push_pull_together

Feature under test: F10 — Simultaneous read/write: a read and write accepted on the same clock edge both take effect, occupancy unchanged, neither flag glitches. F6 — FIFO ordering and count preserved: every word written is read out once and only once, count out equals count in, nothing lost, duplicated, or created.

Description: Verify concurrent read and write at half occupancy: occupancy is preserved and order maintained.

Preconditions: FIFO contains 8 words: values 0 to 7 (preloaded by 8 writes). empty = 0, full = 0.

Stimulus: Assert wr_en = 1 and rd_en = 1 together for 4 consecutive cycles with wr_data = 0x2000 + i for i = 0 to 3. Sample rd_data one cycle after each read edge. Then drain all remaining words the same way.

Expected results:

- During the 4 simultaneous cycles, empty = 0 and full = 0 (occupancy stays 8).
- The 4 reads during the simultaneous phase return 0,1,2,3 (oldest words, FIFO order), each one cycle after its read edge.
- The final drain returns 4,5,6,7,0x2000,0x2001,0x2002,0x2003 in order.
- empty = 1 after the final drain.

Pass criteria: Test passes if occupancy is unchanged during simultaneous access and total order is preserved.

4.9 » T9 — four_beat_burst

Feature under test: F2 — Write/read datapath under a burst: four words written back-to-back. F3 — Registered read latency: back-to-back reads stream one word per cycle, each one cycle behind its accepting edge.

Description: Verify a 4-deep write burst followed by a 4-deep read burst (data integrity, small burst).

Preconditions: FIFO is empty.

Stimulus: Write a burst of 4 words with wr_data = 0xB0,0xB1,0xB2,0xB3 (one per cycle, wr_en = 1). Wait 2 idle cycles. Read a burst of 4 with rd_en = 1 held 4 cycles, sampling rd_data one cycle after each read edge.

Expected results:

- empty = 0 after the first write of the burst.
- full = 0 throughout the test (max occupancy 4 of 16).
- The 4 sampled read values are 0xB0,0xB1,0xB2,0xB3 in order.
- empty = 1 on the edge committing the 4th read; rd_data holds 0xB3 afterwards.

Pass criteria: Test passes if all 4 burst values are returned in order and empty asserts after the drain.

5. Notes for checking

Expected results in this plan are the golden records for the AI checking phase. Testbench logs use the structured format: [CHK] <field>=<value> ... so that each expected-result bullet maps to one or more [CHK] log lines. All data values are hexadecimal unless stated otherwise. Occupancy refers to the number of words currently stored (writes committed minus reads committed). Remember the global sampling rule: read data is valid one cycle after the read request is accepted.